A Reproducible Hybrid Interpretable Framework for Credit-Card Fraud Detection

Shravya Reddy Nalla and Atanu Mukhopadhyay
Shravya Reddy Nalla: PhD Researcher in Artificial Intelligence, University of the Cumberlands
Atanu Mukhopadhyay: Independent Researcher

Abstract. This paper presents a reproducible evaluation of an interpretable fraud-detection framework combining Automated Decision-Rule Generation (ADRG), RuleFit, and a population-level composite z-score layer. Unlike the supplied draft, the revised study uses a public anonymized credit-card transaction benchmark and distributes the exact experiment code, data-acquisition instructions, result files, and checksums. The benchmark contains 284,807 transactions and 492 fraud cases. A chronological 60/20/20 train-validation-test split is used, with five seeded reruns for stochastic models. On the held-out test window, ADRG obtains mean precision 0.753, recall 0.733, and F1 0.739; RuleFit obtains 0.876, 0.715, and 0.786; random forest obtains 0.888, 0.760, and 0.819. The results support the value of interpretable rules as a high-precision alternative, while also showing that the proposed hybrid does not improve over ADRG on this benchmark. We therefore present the method as a reproducible research prototype rather than a production deployment claim.

Keywords: credit-card fraud detection; interpretable machine learning; rule generation; RuleFit; anomaly detection; reproducibility

1. Introduction

Fraud detection is a highly imbalanced classification problem in which operational usefulness depends on both discrimination and the number of alerts requiring investigation. Black-box scores can be effective, but rules and concise explanations remain important when analysts must audit decisions, change policy, or justify an alert. This work revises an earlier manuscript whose numerical tables were based on an unavailable synthetic benchmark. The revision replaces those results with a rerun on a public dataset and makes the computational path explicit.

The contribution is deliberately scoped. We provide an implementation of ADRG, a RuleFit-style sparse rule model, and a population-level z-score baseline; compare them with logistic regression, random forest, and histogram gradient boosting; and publish the exact split, seeds, thresholds, result CSVs, and data checksum. We do not claim that the public benchmark represents a bank's current fraud population, nor do we claim production latency, regulatory approval, or customer-level personalization.

2. Related Work

Tree ensembles and gradient boosting are common baselines for fraud classification [1, 2]. RuleFit combines tree-derived rules with sparse linear terms to provide a more compact representation [3]. Decision-tree paths are also a natural source of human-readable conditions [4]. Statistical anomaly scores offer a complementary view when labels are incomplete or delayed [5]. Class imbalance requires care in both training and evaluation; precision-recall metrics are especially informative when the positive class is rare [6].

3. Public Dataset and Reproducibility Protocol

The evaluation uses the public credit-card fraud dataset distributed through the TensorFlow imbalanced-classification tutorial and associated with the Worldline/ULB benchmark. The raw file has

284,807 rows, 492 fraud labels, and the columns Time, V1–V28, Amount, and Class. V1–V28 are anonymized principal components. We add hour = floor(Time/3600) mod 24 as a deterministic feature. For the z-score layer, the population features are V1–V28 and log1p(Amount). No customer identifier is available in this dataset; consequently, the revised paper does not report customer-specific or personalized z-scores.

Rows are kept in chronological order and split into 170,884 training rows (360 frauds), 56,961 validation rows (57 frauds), and 56,962 test rows (75 frauds). Supervised models use all positive training examples and at most 250,000 randomly sampled negative training examples. The random seeds are 0, 1, 2, 3, and 4. Thresholds are selected using validation data only; the test window is used once for final metrics.

The acquisition URL is https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv. The expected SHA-256 of creditcard.csv is 76274b691b16a6c49d3f159c883398e03ccd6d1ee12d9d8ee38f4b4b98551a89. The reproduction package includes a downloader that verifies this checksum, a pinned environment record, the implementation, the exact rerun command, and the generated CSV outputs.

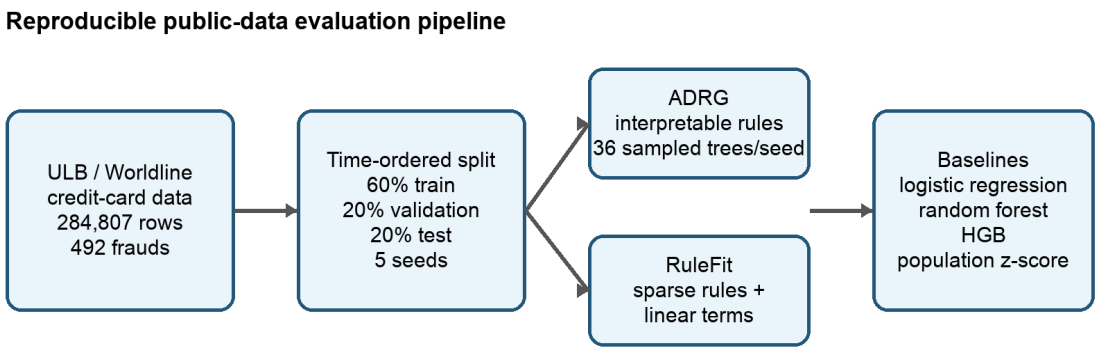


Figure 1. Reproducible evaluation architecture used for the public-data rerun.

4. Methods

ADRG. For each of six positive-to-negative sampling ratios (1, 2, 3, 5, 8, and 10), six seeded decision-tree configurations are trained on the training window. Fraud-positive leaves are converted into simplified conjunctions. Candidate rules are filtered on validation data using at least five alerts, precision at least 0.15, and false-positive rate at most 0.08. A greedy coverage pass retains rules whose marginal alerts retain the minimum precision. Each seed therefore generates 36 trees before filtering.

RuleFit. A 100-tree random forest with maximum depth 3 provides path indicators. Duplicate paths are removed, and the path indicators are combined with winsorized and standardized original features. L1-regularized logistic regression selects the regularization value by validation average precision. The reported number of rules is the number of nonzero selected rule terms.

Anomaly and supervised baselines. The z-score layer estimates means and standard deviations from legitimate training rows and uses the root-mean-square of absolute feature z-scores. We report both a validation-tuned threshold and a fixed threshold of 2.5. Logistic regression is standardized and

class-weighted. Random forest uses 200 trees, minimum leaf size 2, and balanced subsampling. Histogram gradient boosting uses 300 iterations, learning rate 0.05, class weighting, and no early stopping.

5. Results

Model	Precision	Recall	F1	FPR	Alerts	AUPRC	AUROC
Z-score (val-tuned)	0.078	0.427	0.133	0.00661	408.0	0.053	0.955
Z-score (|z|>=2.5)	0.063	0.587	0.113	0.01157	702.0	0.053	0.955
Logistic regression	0.895	0.680	0.773	0.00011	57.0	0.747	0.984
ADRG	0.753	0.733	0.739	0.00034	74.2	0.661	0.867
Hybrid (ADRG OR Z)	0.722	0.733	0.725	0.00039	77.2	—	—
RuleFit	0.876	0.715	0.786	0.00014	61.4	0.776	0.983
Random forest	0.888	0.760	0.819	0.00013	64.2	0.813	0.946
Gradient boosting (HGB)	0.883	0.707	0.785	0.00012	60.0	0.788	0.977

Table 1. Held-out test metrics. Values are means across seeds 0–4 where applicable; deterministic baselines have no seed variation. FPR is shown as a rate. The complete per-seed values are included in results/ULB_results.csv.

For the stochastic models, the sample standard deviations of precision, recall, and F1 are respectively 0.094, 0.042, and 0.045 for ADRG; 0.086, 0.042, and 0.044 for the hybrid; 0.054, 0.020, and 0.020 for RuleFit; and 0.015, 0.000, and 0.006 for random forest. Gradient boosting is deterministic in these recorded reruns apart from runtime.

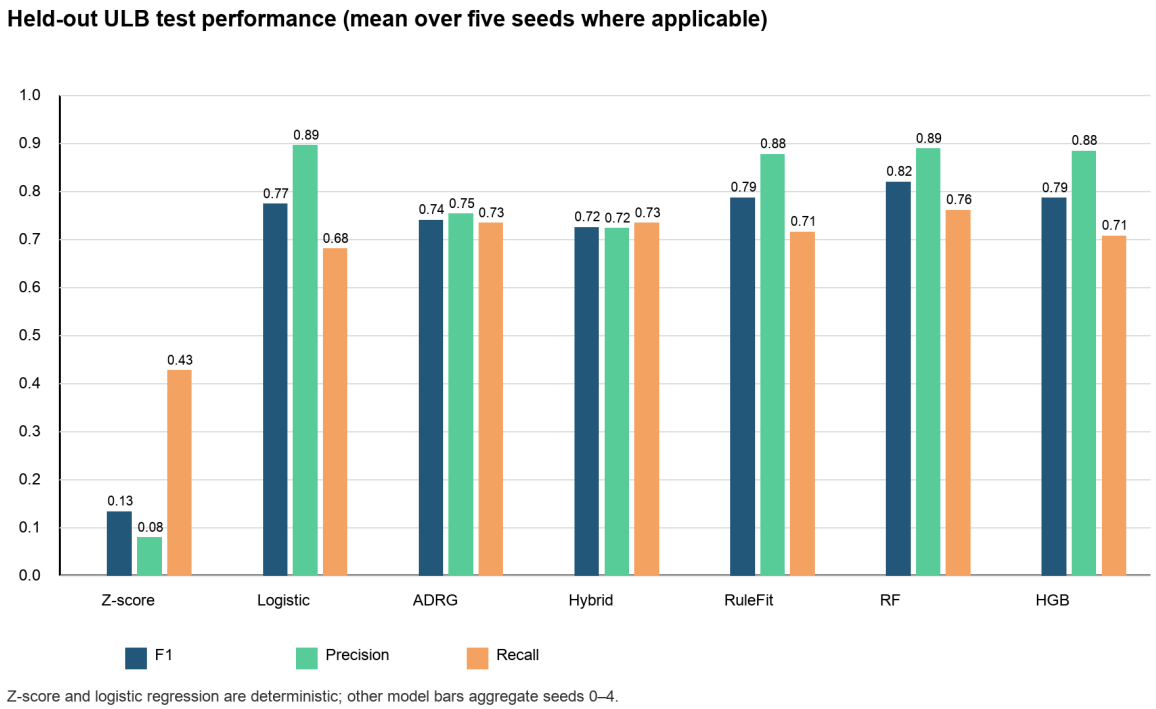


Figure 2. Precision, recall, and F1 on the held-out chronological test window.

ADRG averages 55 of the 75 test frauds while producing 74.2 alerts, with mean precision 0.753 and F1 0.739. RuleFit is more precise on average (0.876) but has lower recall (0.715). Random forest has the highest recall among the principal supervised methods (0.760) and the highest mean F1 (0.819). The hybrid OR rule does not add true positives over ADRG in the recorded reruns and slightly lowers precision, so the public benchmark does not support a claim that the hybrid is superior.

The ADRG model selects 8–13 rules per seed (mean 10.0 ± 2.1) after validation filtering. Its measured fitting time is 0.203 ± 0.047 seconds on the recorded single-thread run. RuleFit selects 22.0 ± 11.5 rules and takes 127.8 ± 11.4 seconds under the same environment. These runtimes are implementation- and hardware-specific and should not be interpreted as deployment capacity.

6. Limitations and Data Use

The dataset is anonymized and public, but it is not a complete representation of modern banking operations. It has no customer identifier, merchant context, intervention cost, concept-drift annotation, or production feedback loop. The chronological split reduces leakage relative to a random split, but the public dataset's time ordering is not a guarantee of future-stationarity. The z-score baseline is population-level rather than customer-personalized. Finally, the ADRG thresholds are research choices; different alert budgets or fraud costs would lead to different selections.

The raw data are not redistributed in the arXiv upload because the package is designed to acquire the public file directly and verify its checksum. Users should review the dataset host's terms before redistribution. The reproduction package contains the acquisition script and the exact hash rather than silently substituting another copy.

7. Reproducibility and Availability

The accompanying reproducibility ZIP contains the final PDF, editable DOCX, figures, source implementation, result CSV files, dataset README, acquisition script, environment versions, and a machine-readable manifest. From a clean environment, the intended command is: python code/run_experiments.py data results ulb. The code accepts either creditcard.csv or an equivalent ulb.parquet conversion. All reported tables in this revision were generated from the supplied ULB result CSVs, and the script records the split counts and model settings in ULB_info.json.

8. Conclusion

This revision turns the manuscript into a reproducible public-data study. Interpretable ADRG rules achieve high precision and competitive recall, while RuleFit and tree ensembles provide useful complementary baselines. The results also show why the hybrid claim must be evaluated rather than assumed: on this benchmark, the OR combination does not improve over ADRG. The principal value of the package is therefore an auditable experimental baseline that others can rerun, criticize, and extend.

9. AI-tool Disclosure

Generative AI tools were used for language editing, document preparation, and reproducibility-package organization. The authors reviewed the manuscript, code, data provenance, references, and reported results, and take responsibility for the accuracy and integrity of the submission. No generated data were used in the experiments.

References

[1] V. Vapnik, The Nature of Statistical Learning Theory, Springer, 1995.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," Proc. KDD, 2016.

[3] J. H. Friedman and B. E. Popescu, “Predictive learning via rule ensembles,” Annals of Applied Statistics, vol. 2, no. 3, pp. 916–954, 2008.

[4] L. Breiman, J. Friedman, R. Olshen, and C. Stone, Classification and Regression Trees, Wadsworth, 1984.

[5] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” ACM Computing Surveys, vol. 41, no. 3, 2009.

[6] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLoS ONE, vol. 10, no. 3, 2015.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[8] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, “Learned lessons in credit card fraud detection from a practitioner perspective,” Expert Systems with Applications, vol. 41, no. 10, pp. 4915–4928, 2014.

[9] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, “Calibrating probability with undersampling for unbalanced classification,” Proc. IEEE SSCI, pp. 159–166, 2015.

[10] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you? Explaining the predictions of any classifier,” Proc. KDD, 2016.

[11] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” Proc. NIPS, 2017.

[12] A. Géron, “Credit card fraud detection,” TensorFlow Tutorial, public data and workflow, accessed 2026.

[13] Worldline and Université Libre de Bruxelles, “Credit card fraud detection dataset,” public anonymized benchmark distributed through the TensorFlow tutorial.